Desafio Desenvolvimento
Relatório

Daniel José Naslavsky Aguiar PS 2026.2

Síntese: Este documento foi manualmente redigido e faz parte do desafio de desenvolvimento, tendo como objetivo fornecer um relatório sobre a construção da API da loja de plantas Folhagem & Cia. Ele detalha: **o que foi feito**, **como**, **as principais dificuldades** e **uso de IA**. A parte da **documentação técnica completa da API** está no arquivo DOCUMENTACAO.md, também presente no repositório.

O que foi feito?

Todas as 7 rotas obrigatórias foram implementadas conforme as 7 regras do negócio.
Abaixo, eu fiz uma tabela para facilitar a visualização do que foi feito:

ROTA	O QUE FAZ?	IMPLEMENTADO
POST /plantas	Cria uma planta	SIM
GET /plantas	Lista as planta, com filtros adicionais	SIM
GET /plantas/:id	Busca uma planta no banco de dados	SIM
PATCH /plantas/:id	Atualiza as informações de uma planta	SIM
DELETE /plantas/:id	Remove uma planta do banco de dados	SIM
POST /plantas/:id/venda	Registra a venda de uma planta	SIM
GET /plantas/estoque/valor-total	Retorna o valor total de estoque e a quantidade de plantas (vasos) total	SIM

Além dos obrigatórios, eu também fiz os bônus:

ROTA/COMANDO	O QUE FAZ?	IMPLEMENTADO
GET /plantas?nome=	Busca por nome	SIM
GET /plantas?ordem=	Ordena em ordem crescente	SIM

GET /plantas/esgotadas	Lista as plantas esgotadas (com estoque zerado)	SIM
yarn seed	Carrega a seed gerada do seed.ts	SIM

Os detalhes de cada rota estão na documentação técnica da API em DOCUMENTACAO.md

Como foi feito?

- **Preparação:** Primeiro, li os dois documentos base do desafio, o PDF "Folhagem & Cia: Desafio de Back-end" e o `DESAFIO.md`, para entender o que era pedido. Em seguida, segui a preparação de ambiente do `README.md` do treinamento de CRUD do CITi e fiz um primeiro teste no Thunder Client criando um usuário, o que confirmou que eu já podia começar o projeto de fato.
- **Prisma Model e Migration:** Comecei adicionando o model `Planta` e o enum `Luminosidade` no `schema.prisma`, tomando como base o model `User` que já existia no treinamento. Depois, gerei a migration, que criou a tabela no banco PostgreSQL. Eu segui o princípio do enum Conta do User para o enum Luminosidade da Planta.
- **Repository:** Depois, criei o PlantRepository, no mesmo modelo do UserRepository, e as respectivas funções de CRUD, que são: createPlanta, findAllPlantas, findPlantaById, updatePlanta e deletePlanta. Eu escolhi criar as funções de repositório primeiro porque o Controller depende do Repository, além de que era mais fácil de entender o padrão por me basear no UserRepository. As funções sellPlanta, getValorTotalEstoque e findPlantasEsgotadas foram criadas depois.
- **Validação:** Também uniformizei a validação da entrada em uma única função, validarPlanta, no PlantController.ts. Ela recebe o corpo da requisição e um parâmetro que indica se é um cadastro: no `POST /plantas`, todos os campos são obrigatórios, e no `PATCH /plantas/:id` só os campos enviados são validados, mas todos seguem as mesmas regras. Escolhi uma função só porque isso me permitiu reaproveitar as mesmas regras em mais de uma rota, mudando apenas esse parâmetro. Além disso, também facilitou no manejo de erros, pois as mensagens de erro ficam todas nessa função validarPlanta, mas são repassadas separadamente dependendo do tipo de erro.
- **Cadastro**: Como agora eu conseguia fazer a validação, criei a função createPlanta em PlantController.ts, que chama validarPlanta internamente e verifica se a entrada é válida, isso é um exemplo do reaproveitamento que falei. Além disso, eu liguei o URL a createPlanta no arquivo routes.ts, seguindo o padrão do CRUD User.
- **Busca e listagem (com filtros)**: Também no PlantController.ts, criei as funções findAllPlantas e findPlantaById, que, respectivamente, listam todas (com filtro opcional e ordenação) e buscam por meio do ID. A findPlantaById verifica, por meio da chamada de idValido, se o ID é válido, e também se a planta existe, e aí retorna os dados da planta. Ela lê da URL os filtros luminosidade, petFriendly, nome e a ordem. Como como texto, converti "true" e "false" para boolean e reaproveitei a validarPlanta para checar os filtros, validando a ordem à parte (só preco ou nome).
- **Atualização e deleção da planta:** Criei a função updatePlanta em PlantController.ts, a qual começa checando se o ID é válido e verifica as entradas do req.body por meio da

função validarPlanta (descrita acima). Também vê se a planta de fato existe,e caso positivo, passa ao PlantRepository.updatePlanta, sem mandar o req.body inteiro para o banco. Como o Prisma ignora valores undefined, os campos que não foram enviados continuam como estavam. Se o ID ou algum dado enviado for inválido, responde 400, se a planta não existir, 404, e se der certo, 200 com a planta atualizada.

Além disso, criei a função deletePlanta, no mesmo arquivo PlantController.ts, que funciona de forma parecida com o deleteUser, verificando se o ID é válido, se a planta existe e, caso positivo, passa o id ao PlantRepository.deletePlanta, sem nada a devolver. Se o ID for inválido, responde 400, se a planta não existir, 404, e se a remoção der certo, responde 204 sem nada a devolver.

- **Valor de estoque e venda de planta:** Primeiro, falando da função getValorTotalEstoque em PlantController.ts, que é utilizado por GET /plantas/estoque/valor-total , a qual busca só os campos preco e estoque de todas as plantas. O controller, por sua vez, chama o PlantRepository.getValorTotalEstoque, que faz essa soma no código com reduce, calculando o valor total em centavos e o total de vasos. Com a loja vazia, o reduce começa em zero, então a resposta é 200 com os dois valores zerados.

  Agora, sobre a função PlantController.sellPlanta, primeiro faço a verificação do ID, e vejo se a quantidade requerida de venda é um número inteiro maior que zero. Com isso, chama-se a função PlantRepository.sellPlanta que internamente verifica se a quantidade é menor ou igual ao estoque, faz a alteração no banco e retorna a quantidade de alterados. A verificação e o desconto acontecem numa única operação no banco, e por isso, se duas pessoas tentarem comprar o último vaso ao mesmo tempo, só uma consegue. Se nada foi alterado, então não foi possível vender a planta, e ocorre um erro 400. Caso ocorra alteração, busca a planta de novo e responde 200 com o estoque atualizado.
- **Busca por nome e ordenação:** Na função PlantController.findAllPlantas há, dentro do req.query, duas variáveis que eu adicionei que são nome e ordem, que permitem ou a busca por nome, que ignora maiúsculas e minúsculas, ou/e a ordenação, que pode ser por nome ou por preco, dependendo da URL. Dentro dessa função, os filtros são passados para o findAllPlantas do PlantRepository, que fica responsável por traduzir esses parâmetros e buscar os dados no banco.
- **Plantas Esgotadas:** criei findPlantasEsgotadas em PlantRepository e no PlantController. O plantController aguarda o URL, chama o PlantRepository.findPlantasEsgotadas, o qual acessa o banco de dados e busca pelas plantas com o estoque zerado (estoque: 0), aí lista para o usuário com código 200.

As principais dificuldades

De começo, eu fiquei muito perdido em relação ao projeto, porque tive dificuldade em entender o que era CRUD e como a API era para ser estruturada. Contudo, a base do CRUD de Usuário disponibilizada me ajudou a ter um primeiro contato, principalmente porque fiquei testando como funcionava, além de tirar dúvidas com Inteligência Artificial.

Falando da construção em si, tive dificuldade na parte da validação de planta, pois não consegui conceber uma função única que pudesse ser chamada pelos outros métodos, e nem da variação de comportamento dependendo de ser um cadastro ou não (POST ou PATCH). Assim, recorri à ajuda do Claude, que me deu exemplos de código, os quais eu adaptei ao projeto e testei. Com isso, aprendi como criar uma função uniforme, função validarPlanta, que muda o comportamento somente com o parâmetro "cadastro".

Além disso, na parte de vender a planta fiquei bastante travado porque não sabia como poderia fazer a verificação de quantidade <= estoque dentro do PlantRepository.sellPlanta, porque já sabia que precisaria fazer a verificação e redução numa única operação. Nesse sentido, não sabia da existência, por exemplo, dos atributos "gte" e "decrement" utilizados no escopo de UpdateMany. Para resolver isso, recorri também ao Claude para me guiar nessa estruturação do código. Logo, aprendi como funcionam os operadores gte e decrement da linguagem TypeScript.

Na parte de listagem da planta, fiquei em dúvida de como poderia fazer a filtragem opcional também, inclusive a ordenação também por meio do atributo "orderBy", então recorri, de novo, à ajuda do Claude para me guiar na estruturação do código. Assim, aprendi que os valores "undefined" são ignorados se não forem passados.

O uso de IA

Sim, eu usei bastante Inteligência Artificial, como Claude e Gemini ao decorrer do desafio. Primeiramente, o "DOCUMENTACAO.md" que contém todo o detalhamento técnico da API foi totalmente gerado pelo Claude, com base no meu código, porque julguei que era um trabalho mecânico que não me agregaria valor de aprendizado. Depois, testei os exemplos e as mensagens de erro contra a API e corrigi o que estava diferente.
Além disso, eu também utilizei IA para resolução de dúvidas, entendimento de erro e geração direta de código. O Claude me deu exemplos de código, que adaptei ao meu projeto e testei: um rascunho do model Planta e do enum Luminosidade, a função validarPlanta, o PlantRepository.sellPlanta (com gte e decrement), o where e o orderBy da listagem com a busca por nome, a getValorTotalEstoque, a findPlantasEsgotadas, a updatePlanta e o script de seed. Ela também me ajudou a entender erros de ambiente, como o erro de conexão com o banco no yarn migration, e a organizar os commits no Git.
Eu conferi o que a IA me respondia, rodei os testes da tabela do PDF no Thunder Client, usei o yarn typecheck e comparei com o CRUD de usuário. Aprendi com isso porque primeiro tentei entender o código que ela gerou e só depois li a explicação, o que me ajudou a entender a lógica por trás.

Daniel José Naslavsky Aguiar PS 2026.2